

Laboratorio di Programmazione

Lezione 2

Operatore di Assegnamento

- Si utilizza il **simbolo =**
- Il valore alla destra di = viene assegnato alla variabile a sinistra di =
- Esempi:

```
num = 57;  
valore = 34,5;  
risp = 's';  
  
num = valore;
```

Operatore cast

```
int num1, num2;
```

```
float media;
```

```
media = (float) (num1 + num2) / 2;
```

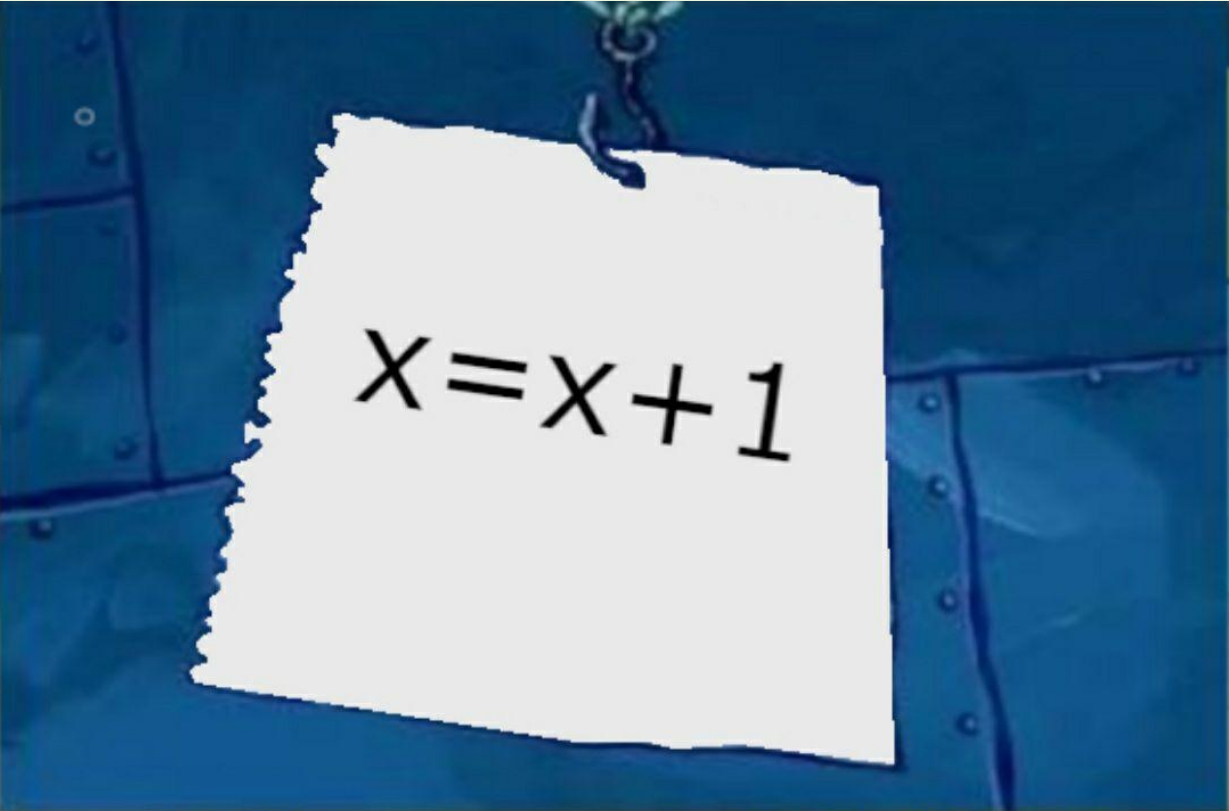
Applica una conversione di tipo.

Operatori Aritmetici

- **Espressioni aritmetiche** sono formate usando gli operatori **+**, **-**, *****, **/**, **%** insieme a variabili e numeri (*operandi*). Il risultato è un numero.
 - Se entrambi gli operandi sono dello stesso tipo, il risultato è di quel tipo.
 - Le operazioni *****, **/**, **%** hanno priorità rispetto a **+**, **-**.
 - L'ordine di priorità può essere alterato usando le parentesi **(,)**.

- Esempi:

```
num = (57 + 3) * 23 ;  
num = num +1 ;
```



$x = x + 1$



Programmers

Mathematitians

Operatori di uguaglianza e relazionali

- Sono operatori che applicati a variabili di "tipo numerico" (o `char`) danno come risultato un valore booleano **true** o **false**: `<`, `>`, `<=`, `>=`, `==`, `!=`
- Esempi:

```
num > 5;  
n1 + n2 >= 14;  
valore == n1;  
continua = val==3;
```

Operatori booleani

- Sono operatori che applicati a variabili di tipo booleano (**true** o **false**) restituiscono un valore booleano:

&& (and), **||** (or), **!** (not)

- Esempi:

```
(num > 5) && (i!=0) ;  
!(n1 + n2 >= 14) ;  
n1 || n2 ;  
val = risp && (r=='s') ;
```

ATTENZIONE:

- In C non esiste il tipo **boolean**, ogni numero non 0 è interpretato come **true** mentre lo 0 corrisponde a **false** .
- Ad esempio:

num && (a<5)

è equivalente a

(num!=0) && (a<5)

Espressioni booleane (condizioni logiche)

- Sono espressioni composte da variabili, espressioni aritmetiche e operatori relazionali, di uguaglianza e booleani che hanno come risultato un valore logico (**true** o **false**) .
- Si utilizzano nelle **strutture di controllo condizionali** e **iterative**.

Regole di precedenza

Più alta

- Operatori *unari* $+$, $-$, $!$
- Operatori aritmetici *binari* $*$, $/$, $\%$
- Operatori aritmetici binari $+$, $-$
- Operatori relazionali $<$, $>$, $<=$, $>=$
- Operatori di uguaglianza $==$, $!=$
- Operatore booleano $\&\&$
- Operatore booleano $||$
- Operatore di assegnamento $=$

Più bassa

Esempi

```
int x = 5;
```

```
int y = 10, z = 15;
```

```
b = (x < 5 && y > x);
```

```
b = (x < 5 || y > x);
```

```
b = (x > 3 || y < 10 && z == 15);
```

```
b = (! (x > 3) && x != z || x + y == z);
```

L'istruzione **if**

- Sintassi

```
if ( <valore booleano> )  
    <istruzione_1>  
    <istruzione_2>  
    <istruzione_3>  
    . . . . .
```

- Significato

<istruzione_1> viene eseguita soltanto se
<valore booleano> è *true*

Esempio

- Visto oggi in laboratorio :

```
if (a < b)
    min=a;
if (b < a)
    min=b;
printf("il minimo e' %d",min);
.....
.....
```

Istruzioni composte (blocchi)

- Quando un gruppo di istruzioni è racchiuso tra coppia di parentesi graffe ({ }), esse formano una unica *istruzione composta*.
- Sintassi:

```
{  
    Istruzione_1  
    Istruzione_2  
    ...  
}
```

Esempio

- Uso istruzione composta per eseguire diverse istruzioni dentro un **if**

```
if (cont < 3)
{
    totale = 0;
    printf("tutto sbagliato!");
}
prezzo = 50;
.....
.....
```

L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>  
    .....  
    <istruzione_r>  
}  
  
<istruzione_x>  
<istruzione_y>  
.....
```

- Significato

Il blocco <istruzione_1>..*<istruzione_r>* viene eseguito **soltanto** se <espressione booleana> è *true*

Attenzione.... Errori in vista!

```
if ( num1 = num2 )  
    printf( "%d e' uguale a %d\n", num1, num2 ); NO!
```

```
if ( num1 == num2 );  
    printf( "%d e' uguale a %d\n", num1, num2 ); NO!
```

```
if ( num1 == num2)  
    printf( "%d e' uguale a %d" \n, num1, num2 ); NO!
```

```
if ( num1 == num2)  
    printf( "&d e' uguale a &d \n ", num1, num2 ); NO!
```

```
if ( num1 == num2 )  
    printf( "%d e' uguale a %d\n", num1, num2 ); OK!
```

L'istruzione **if...else**

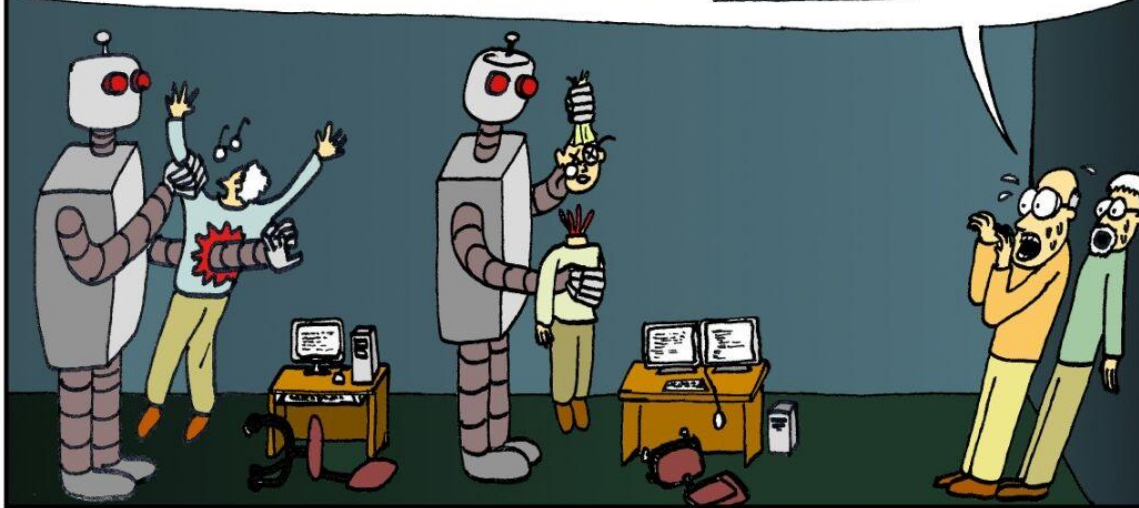
- Sintassi

```
if ( <espressione booleana> )  
    { <istruzione_1>;  
      .....  
      <istruzione_r>; }  
else  
    { <istruzione_r+1>;  
      .....  
      <istruzione_s>; }  
<istruzione_x>;  
<istruzione_y>;  
.....
```

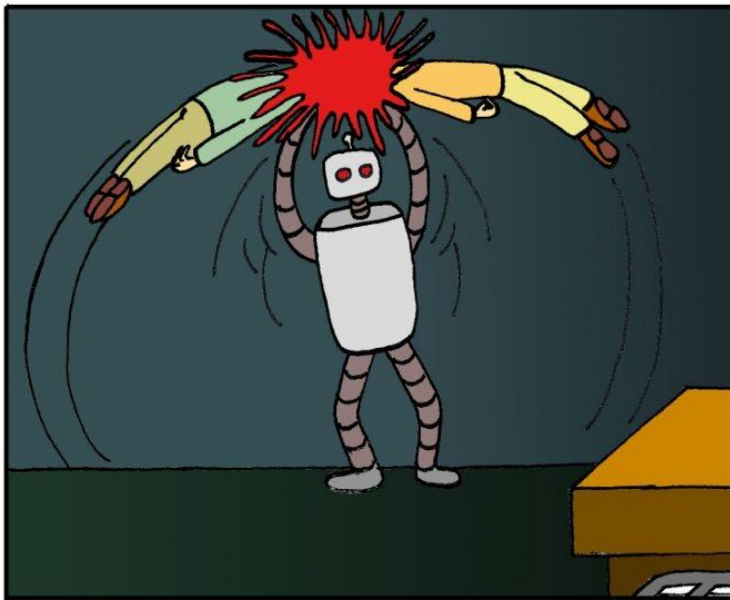
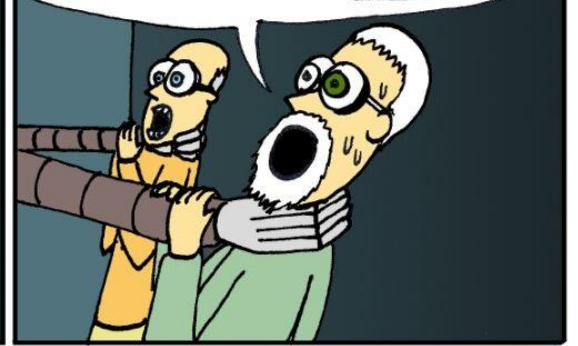
- Significato

SE <espressione booleana> è **true** viene eseguito Il blocco
<istruzione_1>..*<istruzione_r>* **ALTRIMENTI** viene eseguito Il blocco
<istruzione_r+1>..*<istruzione_s>;*

OH NO! THE ROBOTS ARE KILLING US!!!



BUT WHY?!? WE NEVER PROGRAMMED THEM TO DO THIS!!!



```
main() {  
    int isCrazyMurderingRobot=0;  
  
    if (isCrazyMurderingRobot=1)  
        kill(humans);  
    else  
        be_nice_to(humans);  
}
```

Esercizio

```
if (a%3==0 || a%2==0)
    printf("%d", 2*a);
else
    printf("%d", 5*a);
```

- Se a=10 il programma stampa ?
- Se a=42 il programma stampa ?
- Se a=18 il programma stampa ?
- Se a=25 il programma stampa ?

Istruzione **while**

- Sintassi

```
while (<espressione booleana>){  
    <istruzione_1>  
    ...  
    <istruzione_r>  
}  
  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..**<istruzione_r>** viene eseguito **fintanto che** <espressione booleana> è **true**

Iterazione definita : ciclo con "contatore"

numero di iterazioni conosciuto a-priori

- Utilizzo una variabile per contare:
 - La inizializzo a 0 prima del ciclo
 - All'interno del ciclo la incremento
 - La condizione del while controlla quando arrivo al valore scelto

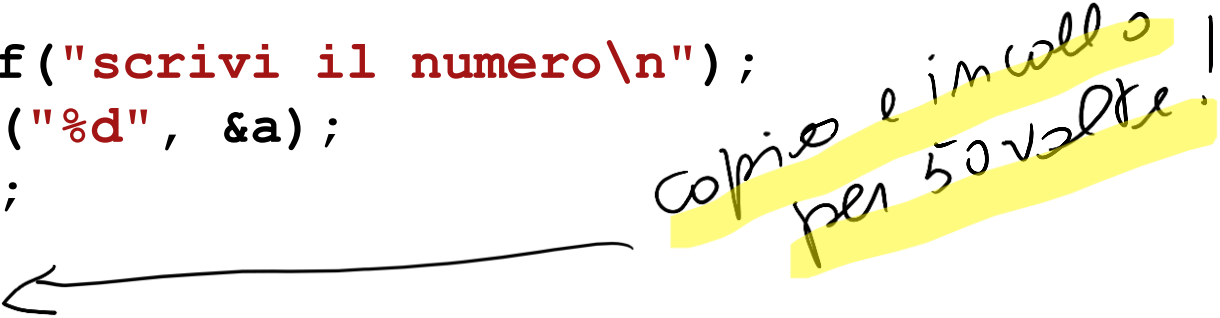
```
cont =0;
while (cont < 5)
{
    printf("finalmente una ripetizione! \n");

    cont= cont+1;
}
printf("tutto bene?");

.....
```

Esercizio: Calcolare la somma di 50 numeri immessi dall'utente.

```
int a, s;  
s=0;  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
.....  
.....  
printf("la somma dei numeri è %d", s);
```



Esercizio : Calcolare la somma di 50 numeri immessi dall'utente.

muovo!

```
int a, s, c;
```

```
s=0;
```

```
c=0;
```

```
while( c < 50 ){
```

```
    printf("scrivi il numero\n");
```

```
    scanf("%d", &a);
```

```
    s=s+a;
```

```
    c=c+1;
```

```
}
```

```
printf("la somma dei numeri è %d", s);
```

c è il mio contatore

Un ciclo a iterazione "indefinita"

numero di iterazioni NON conosciuto a-priori

- La condizione del while determina il numero di volte che il ciclo viene eseguito .. (può dipendere dalla particolare esecuzione)

```
c=1;
scanf ("%d", &num) ;
while (num !=33)
{
    printf(" \n No, non hai indovinato ...);
    c=c+1;
    printf(" \n Inserisci un altro numero :");
    scanf ("%d", &num) ;
}
printf("hai indovinato il mio numero in %d
tentativi", c) ;
```

Esercizio: Calcolare la somma di vari numeri immessi dall'utente; l'immissione termina quando l'utente inserisce 0

```
int a, s;

printf("scrivi il numero\n");
scanf("%d", &a);
s=0;

while( a != 0 ){
    s=s+a;
    printf("scrivi il numero\n");
    scanf("%d", &a);
}

printf("la somma dei numeri è %d", s);
```

Attenzione.... Errori in vista!

```
int num = 5;
while( num > 3 ){
    printf("hello world");
}
```

NO

```
int num = 5;
while( num > 3 ){
    printf("hello world");
    num=num+1;
}
```

NO

```
int num = 5;
while( num < 7 ){
    printf("hello world");
    num=num+1;
}
```

OK

Istruzione **while**

ATTENZIONE!!!

Tra le istruzioni del corpo del **while** ci deve essere qualcosa che coinvolge e modifica le variabili della *<espressione booleana>* e la rende false

Altrimenti... LOOP INFINITO